

LECTURE NOTES

Algorithm and Programming

Structures and Union

Pandu Dwi Luhur Pambudi, S.Kom., M.Kom., M.I.M



Learning Outcomes

1. LO2: Analyze programs to solve problems using the C language and AI problem-solving sessions related to the topics.

Outline Materi

1. Structure
2. Typedef
3. Union
4. Bit Field
5. Enumeration
6. Problem-Solving Session with AI: Analysis of Composite Data Types

7.1

Structure

Structure atau struct merupakan struktur data yang menggabungkan beberapa data yang berbeda tipe (heterogen) tetapi data tersebut saling berkaitan. Misalnya data mengenai StudentId, nama, dan IPK seorang mahasiswa. Ketiga data ini memiliki tipe data yang berbeda tetapi masih saling berhubungan yaitu data akademik seorang mahasiswa. Dengan menggunakan struct maka data ini bisa diolah per-elemen (per field) atau secara keseluruhan (per struct, per record).

Untuk mendefinisikan struktur, Anda harus menggunakan pernyataan struct. Pernyataan Struct mendefinisikan tipe data baru, dengan lebih dari satu anggota. Format pernyataan struct adalah sebagai berikut -

Berikut contoh sintaks struct pada bahasa C :

```
struct [type_name]

{

    member definition; member definition;

    ...

    member definition;

} [one or more structure variables];
```

Type_name : adalah nama tipe struct.

Member definition: adalah nama member atau field.

one or more structure variables: adalah nama variabel struct. Berikut adalah beberapa contoh deklarasi struct pada bahasa C :

```
//deklarasi variabel struct.
struct {
    char StudentId[11];
    float ipk;
    char nama[50];
}mhs;

//deklarasi tipe struct dan variabel secara bersamaan.
struct data{
    char StudentId[11];
    float ipk;
    char nama[50];
}mhs;
```

```
//deklarasi tipe struct dan variabel struct secara terpisah.  
struct data{  
    char StudentId[11];  
    float ipk;  
    char nama[50];  
};  
  
struct data mhs;
```

Pada beberapa compiler C, keyword struct pada deklarasi variabel struct secara terpisah dapat dihilangkan. Seperti tipe data yang lain, sebuah variabel struct dapat diinisialisasi diawal pada saat pendeklarasian. Berikut contoh inisialisasi struct pada C:

```
#include<stdio.h>  
  
struct data{  
    char StudentId[11];  
    float ipk;  
    char nama[50];  
};  
  
int main()  
{  
    struct data mhs = {"1900012223", 3.78, "Adam Syahputra"};  
    printf("StudentId : %s\n", mhs.StudentId);  
    printf("IPK : %.2f\n", mhs.ipk);  
    printf("Nama : %s\n", mhs.nama);  
  
    getchar();  
    return 0;  
}
```

Untuk mengakses field struct gunakan dotoperator yaitu nama variabel struct diikuti titik dan nama field. Berikut adalah hasil output dari contoh inisialisasi sturct diatas :

```
StudentId : 1900012223  
IPK : 3.78  
Nama : Adam Syahputra
```

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Dalam pengembangan program nyata, kapan sebaiknya kita menggunakan `struct` daripada hanya menggunakan variabel biasa?

Jelaskan dengan contoh situasi yang tepat!

Hint:

Pikirkan contoh data yang memiliki beberapa elemen berbeda jenis (misalnya data mahasiswa, data kendaraan, data karyawan) yang selalu digunakan bersama. Menggunakan `struct` membuat program lebih terstruktur, mudah diolah per field atau per record.

Pertanyaan Diskusi:

Bagaimana cara mengakses dan mengubah isi dari sebuah field pada `struct`?

Berikan contoh sintaks pendeklarasian, inisialisasi, dan akses field menggunakan `dot operator`.

Hint:

Perhatikan format:

```
struct Mahasiswa {  
    int id;  
    char nama[50];  
    float ipk;  
};  
struct Mahasiswa mhs;  
mhs.id = 12345;  
mhs.ipk = 3.8;
```

Diskusikan juga perbedaan inisialisasi langsung vs. pengisian field satu per satu.

Pertanyaan Diskusi:

Apa keuntungan menggunakan `struct` untuk menangani data kompleks?

Apakah ada keterbatasan `struct` dalam bahasa C yang perlu diwaspadai?

Hint:

Bahas bagaimana `struct` mempermudah pengelompokan data heterogen menjadi satu unit logis. Sebagai kekurangan, diskusikan bahwa `struct` tidak mendukung fungsi bawaan seperti `class` di OOP — sehingga tidak ada inheritance atau encapsulation secara otomatis.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Introduction to Structures in C: [Youtube](#)).

7.2

Typedef

Pada bahasa pemrograman C disediakan keyword typedef, yang dapat digunakan untuk membuat tipe penamaan baru. Misalnya typedef dapat digunakan dengan struct untuk mendefinisikan tipe data baru dan kemudian tipe data ini digunakan untuk mendefinisikan variabel struct, dengan contoh sebagai berikut :

```
#include <stdio.h>
#include <string.h>
typedef struct Books
{
    char title[50];
    int book_id;
} Book;    //variable struct

int main( )
{
    Book book;    //struct name
    strcpy( book.title, "C Programming");
    book.book_id = 6495407;

    printf( "Book title : %s\n", book.title);
    printf( "Book book_id : %d\n", book.book_id);

    getchar();
    return 0;
}
```

Berikut output dari program tersebut adalah :

```
Book title : C Programming
Book book_id : 6495407
_
```

Menggunakan Typedefs dapat membantu membuat program lebih mudah dibaca dan dipelihara. Seringkali, typedef digunakan untuk membuat sinonim untuk tipe bawaan. Misalnya, program yang membutuhkan bilangan bulat empat byte dapat menggunakan tipe *int* pada satu sistem dan tipe *long* pada yang lain. Program yang dirancang untuk portabilitas sering menggunakan typedef untuk membuat alias untuk bilangan bulat empat byte, seperti integer. Untuk port program ke platform lain, Anda dapat dengan mudah mengubah integer typedef dan mengkompilasi ulang program.

Bahan Diskusi



Silahkan diskusikan bersama dosen/tutor/rekan mahasiswa mengenai:

Pertanyaan Diskusi:

Apa manfaat utama menggunakan `typedef` pada program C, khususnya ketika digunakan bersama `struct`?

Jelaskan bagaimana `typedef` membuat kode lebih rapi dan mudah dirawat.

Hint:

Pikirkan bagaimana `typedef` dapat menggantikan deklarasi `struct` yang panjang dengan nama yang lebih ringkas. Contoh:

```
typedef struct {  
    int id;  
    char name[50];  
} Student;
```

Sekarang cukup menulis `Student s1;` daripada `struct Student s1;`.

Pertanyaan Diskusi:

Bagaimana `typedef` dapat membantu membuat program C menjadi lebih portabel antar sistem operasi atau platform berbeda?

Hint:

Bahas contoh membuat alias untuk tipe data seperti `integer` agar mudah diganti di satu tempat saja jika pindah platform, misalnya:

```
typedef long int integer;
```

Kalau pindah platform, cukup ganti `long int` jadi `int` jika perlu.

Pertanyaan Diskusi:

Apakah semua program C perlu menggunakan `typedef`? Kapan penggunaan `typedef` justru bisa membuat program membingungkan?

Hint:

Diskusikan bahwa untuk tipe data sederhana atau program kecil, `typedef` tidak wajib. Terlalu banyak alias juga dapat membuat orang bingung melacak tipe data aslinya.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (C typedef: [Youtube](#)).

7.3

Union

Seperti struktur, union adalah tipe data yang diturunkan, tetapi anggotanya berbagi memori yang sama. Pada waktu yang berbeda selama eksekusi program, beberapa variabel mungkin tidak relevan. Jadi, Union berbagi ruang daripada membuang -buang penyimpanan pada variabel yang tidak digunakan. Anggota union dapat memiliki jenis apa pun. Jumlah byte yang digunakan untuk menyimpan union harus setidaknya cukup untuk menahan anggota terbesarnya.

Dalam kebanyakan kasus, union berisi dua atau lebih item dari berbagai jenis. Anda hanya dapat merujuk satu anggota - dan dengan demikian hanya satu jenis - pada satu waktu. Adalah tanggung jawab Anda untuk merujuk data dengan jenis yang tepat. Merujuk data yang saat ini disimpan dengan variabel dari tipe yang salah adalah kesalahan logika-hasilnya tergantung pada implementasi.

Berikut sintaks dalam penggunaan union pada bahasa C :

```
union [type_name]  
  
{  
  
    member definition;  
  
    member definition;  
  
    ...  
  
    member definition;  
  
    } [one or more union variables];
```

Type_name : adalah nama tipe union.

Member definition : adalah nama-nama variabel yang kongsi memori, minimal memiliki 2 variabel.

one or more union variables : adalah nama variabel union. Berikut contoh penggunaan union pada bahasa C :

Structures and Union

```
#include <stdio.h>
#include <string.h>

union Data
{
    int i;
    float f;
    char str[20];
};

int main( )
{
    union Data data;

    printf( "Memory size occupied by data : %d\n", sizeof(data));

    getchar();
    return 0;
}
```

Pada contoh program tersebut terlihat bahwa memori terbesar adalah charstr sebesar 20 byte maka memori yang dibutuhkan adalah sebesar 20 byte. Dari program tersebut akan menghasilkan output sebagai berikut :

```
Memory size occupied by data : 20
```

Contoh

```
# include <stdio.h>
```

```
void main() {
```

```
    union tbil{
```

```
        unsigned int di;
```

```
        unsigned char dc[2];
```

```
    } bil_x;
```

```
    bil_x.di = 321;
```

```
    printf("di   = %d \n", bil_x.di);
```

```
    printf("dc[0] = %d \n", bil_x.dc[0]);
```

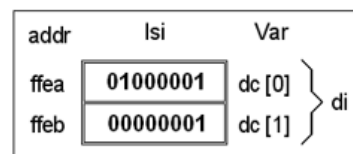
```
    printf("dc[1] = %d\n", bil_x.dc[1]);
```

```
}
```

```
//size assumption for int data type is 2 bytes
```

Hasil :

321= 101000001



```
di    = 321
dc[0] = 65
dc[1] = 1
```

Contoh program untuk mengonversi 1 byte angka desimal (0-255) menjadi sistem biner menggunakan Union dan Bit-Field

```
#include <stdio.h>

struct biner{
    unsigned bit0:1; unsigned bit1:1;
    unsigned bit2:1; unsigned bit3:1;
    unsigned bit4:1; unsigned bit5:1;
    unsigned bit6:1; unsigned bit7:1;
};

int main()
{
    unsigned char ch;
    union byte x;
    printf("Input number (0-255): ");
    scanf("%d",&ch);
    x.ch=ch;
    printf("%d binary = %d%d%d%d%d%d%d%d\n",
    ch,x.bit.bit7,x.bit.bit6,x.bit.bit5,x.bit.bit4,
    x.bit.bit3,x.bit.bit2,x.bit.bit1,x.bit.bit0);
    getch();
    return 0;
}
```

```
union byte{
    unsigned char ch;
    struct biner bit;
};
```

Operasi yang dapat dilakukan pada union adalah:

- menugaskan union ke union lain dari jenis yang sama,
- Mengambil alamat variabel union (&),
- mengakses anggota union melalui operator anggota struktur (.) dan operator pointer struktur (->), dan
- Zero-initialized Union.

Dua union tidak boleh dibandingkan menggunakan operator == dan != Untuk alasan yang sama bahwa dua struktur tidak dapat dibandingkan.

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Apa perbedaan mendasar antara `struct` dan `union` dalam hal alokasi memori dan cara penggunaannya? Jelaskan dengan contoh sederhana.

Hint:

Ingat bahwa pada `struct`, setiap anggota punya ruang memori sendiri-sendiri sehingga total memori adalah jumlah semua anggota. Pada `union`, semua anggota menempati lokasi memori yang sama, hanya satu anggota yang aktif pada satu waktu.

Pertanyaan Diskusi:

Mengapa penggunaan `union` memerlukan kehati-hatian lebih dibanding `struct`? Berikan contoh situasi di mana salah referensi anggota `union` bisa menyebabkan error logika.

Hint:

Diskusikan bahwa programmer harus tahu anggota mana yang sedang valid. Contoh: jika data integer disimpan, jangan tiba-tiba akses sebagai char array tanpa konversi yang tepat.

Pertanyaan Diskusi:

Berikan contoh kasus nyata di mana `union` lebih efisien daripada `struct`. Mengapa dalam kasus tersebut `union` dipilih?

Hint:

Pikirkan kasus seperti protokol jaringan atau perangkat embedded, di mana satu blok memori digunakan untuk menyimpan data dalam format berbeda sesuai kebutuhan (misalnya data sensor bisa berupa integer atau float).

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Introduction to Unions in C: [Youtube](#)).

Bit Field

Anda dapat menentukan jumlah bit untuk menyimpan anggota integral yang unsigned atau signed dari sebuah struct atau union. Dikenal sebagai **bit field**, ini memungkinkan pemanfaatan memori yang lebih baik dengan menyimpan data dalam jumlah minimum bit yang diperlukan. Bit Field biasanya dinyatakan sebagai int atau unsigned int.

Contoh

```
struct bitCard {  
    unsigned int face : 4;  
    unsigned int suit : 2;  
    unsigned int color : 1;  
};
```

Array layar diisi dengan 2000 elemen, di mana setiap elemen menggunakan 2 byte (14 bit membutuhkan setidaknya dua byte)

Bit field dapat mengurangi jumlah memori yang dibutuhkan program, tetapi bergantung pada mesin. Meskipun Bit field menghemat ruang, menggunakannya dapat menyebabkan kompiler menghasilkan kode bahasa mesin yang lebih lambat. Ini terjadi karena dibutuhkan operasi bahasa mesin tambahan untuk mengakses hanya bagian dari unit penyimpanan yang dapat dialamatkan.

Bit field tidak memiliki alamat, jadi berusaha mengambil alamat bit field dengan “&” operator adalah kesalahan. Juga, menggunakan *sizeof* dengan bit field adalah kesalahan.

unnamed bit field digunakan sebagai padding dalam struct. Misalnya,

```
struct example {  
    unsigned int a : 13;  
    unsigned int : 19;  
    unsigned int b : 4;  
};
```

menggunakan field 19-bit yang unnamed sebagai padding. Tidak ada yang bisa disimpan dalam 19-bit itu. Anggota B (dengan asumsi komputer four-byte-word) disimpan dalam kata memori yang terpisah.

Bidang bit yang tidak disebutkan namanya dengan lebar nol menyelaraskan bidang bit berikutnya pada batas unit penyimpanan baru. Misalnya, struct.

```
struct example {  
    unsigned int a : 13;  
    unsigned int : 0;  
    unsigned int : 4;  
};
```

menggunakan unnamed field 0-bit untuk melewati bit yang tersisa (sebanyak yang ada) dari unit penyimpanan di mana A disimpan dan untuk menyelaraskan B pada batas unit penyimpanan berikutnya

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Mengapa Bit Field berguna dalam mendesain program yang efisien dalam penggunaan memori? Berikan contoh situasi nyata di mana Bit Field lebih diutamakan dibandingkan tipe data biasa.

Hint:

Diskusikan bahwa Bit Field memungkinkan kita menyimpan data kecil seperti flag, status, atau register hardware dengan hemat memori. Contoh: pengaturan status bit untuk komunikasi protokol atau kontrol hardware.

Pertanyaan Diskusi:

Apa keterbatasan Bit Field di C yang harus diperhatikan oleh programmer? Mengapa penggunaan operator `&` atau `sizeof` pada Bit Field dianggap kesalahan?

Hint:

Ingat bahwa Bit Field tidak memiliki alamat memori sendiri—mereka hanyalah bagian dari storage unit yang lebih besar. Karena itu, tidak bisa diambil address-nya atau dihitung ukuran bit field-nya secara terpisah.

Pertanyaan Diskusi:

Jelaskan peran unnamed bit field dan unnamed zero-width bit field dalam struct. Bagaimana mekanisme ini membantu alignment memori?

Hint:

Diskusikan bahwa unnamed bit field digunakan untuk padding—menyediakan jarak atau batas yang memisahkan field-field agar disimpan di unit memori yang terpisah. Zero-width bit field memaksa field berikutnya berada di storage unit baru.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Bit fields in C Programming language: [Youtube](#)).

7.5

Enumeration

Enumeration adalah tipe data yang ditetapkan oleh pengguna yang terdiri dari konstanta integral dan setiap konstanta tersebut diberikan nama. Berikut sintaks enum pada bahasa C :

```
enumtype_name{ value1, value2,...,valueN };
```

Type_name : adalah nama dari tipe data enum.

Value : nilai dari enum

Berikut adalah contoh penggunaan enum pada bahasa C :

```
#include <stdio.h>
#include <string.h>

enum week{ sunday, monday, tuesday, wednesday, thursday, friday, saturday};

int main()
{
    enum week today;
    today=wednesday;
    printf("%d day",today+1);

    getchar();
    return 0;
}
```

Berikut hasil dari output program diatas :

```
4 day_
```

Contoh

```
#include <stdio.h>

enum boolean {false, true};

enum boolean evenCheck(int n) {
    enum boolean checkResult;
    if (n % 2 == 0) checkResult = true;
    else checkResult = false;
    return checkResult;
}

int main () {
    int num;
```

```
100
even
```

```
37
odd
```

```
enum boolean result;

scanf("%d", &bil);
result = evenCheck(bil);
if (result == true) printf("even");
else printf("odd");
return 1;
}
```

7.6

Problem-Solving Session with AI: Analysis of Composite Data Types

HAICOLA Section: Mahasiswa dapat menggunakan AI Code Assistant (seperti **Gemini** atau **ChatGPT**) untuk melakukan pengecekan sintaks, memahami error, dan memperbaiki kode C secara efisien.

Dalam pemrograman tingkat lanjut di C, seringkali diperlukan untuk mengelompokkan data yang berbeda tipe menjadi satu unit logis. **struct** dan **union** adalah alat utama untuk tugas ini. Menguasai cara mendefinisikan, mengakses, dan memanipulasi tipe data ini sangat penting untuk **LO 2** karena membantu dalam pemodelan data yang kompleks.

Fokus: Struktur Data Kompleks dan Alokasi Memori

Kesalahan dalam menggunakan tipe data komposit sering berkaitan dengan pemahaman alokasi memori dan cara mengakses anggota:

1. **Akses Anggota (Member Access):** Kesalahan penggunaan operator titik (.) versus operator panah (->) ketika bekerja dengan *struct* atau *union* yang diakses melalui *pointer*.
2. **Pemahaman struct vs. union:** Memahami perbedaan mendasar dalam alokasi memori —di mana **struct** menyimpan semua anggotanya, sedangkan **union** hanya menyimpan satu anggota pada waktu tertentu—yang sangat penting untuk **LO 2**.
3. **Kesalahan Typedef:** Kesalahan sintaks saat mendefinisikan alias baru menggunakan typedef.

Contoh Praktis: Menggunakan AI untuk Analisis Structure dan Union

Anda dapat menggunakan AI Code Assistant (seperti Gemini) untuk meminta alat tersebut **membandingkan** dua tipe data komposit atau **mendiagnosis** kesalahan akses anggota, yang memperkuat pemahaman Anda tentang bagaimana C mengatur data di memori.

Skenario 1: Kesalahan Akses Anggota pada *Struct* dengan *Pointer*

Mahasiswa mencoba mengakses anggota *struct* menggunakan operator yang salah.

Kode yang Bermasalah:

```
#include <stdio.h>

struct Student {
    int id;
    float gpa;
};

int main() {
    struct Student s1;
    struct Student *ptr_s1 = &s1;

    // Kesalahan: Menggunakan operator titik (.) pada pointer
    ptr_s1.id = 101;

    printf("ID: %d\n", ptr_s1->id);
    return 0;
}
```

Permintaan (Prompt) ke AI:

"Saya mencoba mengisi data 'id' dari *struct* yang diakses melalui pointer (*ptr_s1*), tetapi saya mendapat error 'request for member 'id' in something not a structure or union'. Ini kodenya: [masukkan kode di atas]. Apa operator yang benar untuk mengakses anggota melalui pointer?"

Respon AI yang Diharapkan:

AI akan menjelaskan bahwa ketika Anda memiliki **pointer ke struct** (seperti *ptr_s1*), Anda harus menggunakan **operator panah (->)** untuk mengakses anggotanya, bukan operator titik (.). Perbaikan kodenya adalah ***ptr_s1->id = 101;***. AI akan menjelaskan bahwa **.** digunakan untuk variabel *struct* biasa, sedangkan **->** adalah singkatan dari **(*pointer).member**.

Skenario 2: Menganalisis Perbedaan Memori (*struct* vs. *union*)

Permintaan (Prompt) ke AI:

"Jelaskan perbedaan alokasi memori antara *struct* dan *union* di C. Berikan contoh skenario pemecahan masalah di mana saya harus memilih *union* daripada *struct*."

Respon AI yang Diharapkan:

AI akan menjelaskan bahwa **struct** mengalokasikan memori untuk **semua anggotanya secara terpisah**, sehingga total ukurannya adalah jumlah ukuran anggotanya (ditambah padding). Sebaliknya, **union** mengalokasikan memori **hanya untuk anggota terbesarnya**, dan semua anggota berbagi ruang memori yang sama. AI kemudian akan menyarankan **union** untuk kasus di mana Anda memiliki data yang dapat diwakilkan oleh **salah satu dari beberapa tipe data** pada waktu

tertentu (misalnya, merekam nilai yang bisa berupa *int* atau *float*, tetapi tidak keduanya sekaligus). Hal ini membantu **menghemat memori** dan merupakan contoh **pemecahan masalah** menggunakan fitur bahasa C (LO 2).

Praktik Terbaik untuk Mahasiswa

- **Pahami Pemodelan Data:** Gunakan AI untuk memverifikasi apakah *struct* atau *union* adalah pilihan terbaik untuk memodelkan data tertentu, yang merupakan inti dari **LO 2: Menganalisis program....**
- **Validasi Pointer Access:** Jika Anda menggunakan *malloc* atau mengakses *struct* melalui fungsi, gunakan AI untuk memastikan Anda selalu menggunakan operator *->* dengan benar.
- **Latihan Employability Skill:** Menganalisis tipe data C yang kompleks dan memilih *structure* yang paling tepat untuk masalah (efisiensi ruang vs. akses data) melatih kemampuan **Problem-Solving and Decision-Making**.

Bahan Diskusi



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Pertanyaan Diskusi:

Mengapa programmer menggunakan `enum` dalam C padahal bisa saja menggunakan `#define` atau `const int` untuk mendefinisikan konstanta? Apa kelebihan menggunakan `enum`?

Hint:

Diskusikan bahwa `enum` membuat program lebih terbaca, lebih terstruktur, dan memungkinkan debugging lebih mudah karena terkait langsung dengan tipe data. Juga, `enum` memberikan batasan nilai legal, berbeda dengan `#define`.

Pertanyaan Diskusi:

Apa nilai default untuk item pertama, kedua, dan seterusnya dalam `enum` jika programmer tidak menetapkan nilai manual? Bagaimana cara menyesuaikan nilai `enum` agar tidak selalu berurutan?

Hint:

Ingat bahwa secara default, item pertama = 0, kedua = 1, ketiga = 2, dst. Programmer dapat mengatur nilai awal atau loncatan tertentu dengan memberi nilai manual pada salah satu anggota `enum`.

Pertanyaan Diskusi:

Lihat contoh `enum boolean` pada program C di atas. Mengapa menggunakan `enum` sebagai tipe kembalian (return type) dari fungsi `evenCheck` dianggap praktik yang baik?

Hint:

Bahas bahwa memakai `enum` sebagai tipe return membantu menjaga konsistensi hasil (hanya `true` atau `false` yang valid), mencegah error nilai di luar rentang, dan meningkatkan kejelasan maksud program.

Belajar Mandiri



Pelajari secara mandiri terkait dengan materi ini pada PPT dan video yang ada di LMS. Selain itu, rekan-rekan juga dapat menonton video berikut ini (Enumerations in C: [Youtube](#)).

Post-Test



Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 1: Structure

Dalam struct berikut, bagaimana cara yang benar untuk mengakses field `ipk` dari variabel `mhs`?

```
struct Mahasiswa {  
    int studentId;  
    char name[50];  
    float ipk;  
} mhs;
```

- A. `mhs->ipk;`
- B. `mhs.ipk;`
- C. `mhs:ipk;`
- D. `mhs['ipk'];`

Soal 2: Typedef

Apa keuntungan utama menggunakan `typedef` pada struct di C?

- A. Membuat program lebih lambat tapi aman
- B. Membuat tipe data bawaan menjadi tidak valid
- C. Membuat nama tipe baru lebih mudah dibaca dan dipakai
- D. Menambah memori yang digunakan

Soal 3: Union

Perhatikan union berikut:

```
union Data {  
    int i;  
    char str[20];  
} data;
```

Berapa memori yang digunakan oleh `data`?

- A. Ukuran int ditambah ukuran `char[20]`
- B. Ukuran variabel terbesar saja
- C. Hanya ukuran int
- D. Hanya ukuran `char`

Post-Test

Setelah menyelesaikan materi ini, silahkan kerjakan soal berikut ini:

Soal 4: Bit Field

Apa fungsi utama bit field dalam struct di C?

- A. Memperbesar penggunaan memori
- B. Mengurangi efisiensi kode
- C. Menghemat memori dengan menentukan jumlah bit yang digunakan
- D. Menambahkan alamat pada anggota bit

Soal 5: Enumeration

Dalam enum berikut, berapa nilai `GREEN`?

```
enum color { RED, GREEN, BLUE };
```

- A. 0
- B. 1
- C. 2
- D. 3

Kesimpulan

- Structure atau struct merupakan struktur data yang menggabungkan beberapa data yang berbeda tipe (heterogen) tetapi data tersebut saling berkaitan.
- Typedef, yang dapat digunakan untuk membuat tipe penamaan baru. Misalnya typedef dapat digunakan dengan struct untuk mendefinisikan tipe data baru dan kemudian tipe data ini digunakan untuk mendefinisikan variabel struct.
- Union digunakan untuk kongsi (menggabungkan) memori. Dengan menggunakan union, suatu lokasi memori dapat ditempati oleh dua atau beberapa variabel dengan masing-masing tipe data yang berbeda.
- Anda dapat menentukan jumlah bit untuk menyimpan anggota integral yang unsigned atau signed dari sebuah struct atau union. Dikenal sebagai **bit field**, ini memungkinkan pemanfaatan memori yang lebih baik dengan menyimpan data dalam jumlah minimum bit yang diperlukan.
- Enumeration adalah tipe data yang ditetapkan oleh pengguna yang terdiri dari konstanta integral dan setiap konstanta tersebut diberikan nama.

Daftar Pustaka

- Paul J. Deitel & Harvey. Deitel. (2022). C how to program. 09. Pearson Education. Hoboken. ISBN:978-0-13-739839-3. Chapter 10
- Collecting Data Items of Different Types: <http://aelinik.free.fr/c/ch19.htm>
- Structs, Enums, and Unions: <http://www.lysator.liu.se/c/c-faq/c-9.html>
- http://www.tutorialspoint.com/cprogramming/c_structures.htm
- http://www.tutorialspoint.com/cprogramming/c_typedef.htm
- http://www.tutorialspoint.com/cprogramming/c_unions.htm
- <http://www.programiz.com/c-programming/c-enumeration>

Berikut jawaban untuk pertanyaan Post-Test diatas:

1. **Jawaban benar:** B

Alasan: Untuk mengakses anggota struct melalui variabel struct langsung, gunakan dot operator ..

2. **Jawaban benar:** C

Alasan: `typedef` mempermudah penamaan tipe data baru, meningkatkan keterbacaan dan portabilitas.

3. **Jawaban benar:** B

Alasan: Union hanya memerlukan memori sebesar anggota terbesarnya, karena semua anggota share memori yang sama.

4. **Jawaban benar:** C

Alasan: Bit field digunakan untuk menghemat memori dengan menetapkan jumlah bit yang diperlukan.

5. **Jawaban benar:** B

Alasan: Enum dimulai dari 0 secara default, jadi `RED=0`, `GREEN=1`, `BLUE=2`.